META HACKATHON · PITCH DECK

EcoCycle, improved by Meta Muse

EcoCycle is a working waste-to-value platform in Lagos. In this project, Meta Muse studied the existing app, designed new smart features, wrote the code, redesigned the screens, and tested everything.

CODEBASE UNDERSTANDING | FULL-STACK BUILD | UI REDESIGN | TESTED & WORKING

THE HEADLINE

One AI coding partner took a finished app and made it smarter: photo-based waste estimates, vehicles that cannot overfill, and forecasts that tell the facility what arrives tomorrow.

Built with Meta Muse · Muse Code (muse-wonder-1.0) · EcoCycle — Lagos, Nigeria

THE STARTING POINT

EcoCycle before Muse

The app already worked. Households requested pickups, agents collected waste, and the facility paid people based on a real scale. But three problems held it back:

Households guessed their waste weight

People typed a number on a slider. Guesses were often wrong, so agents travelled for loads that were smaller — or bigger — than expected.

Agents could overfill their vehicles

The app showed vehicle capacity, but nothing stopped an agent from accepting more jobs than the vehicle could carry.

The facility planned with one big number

Managers saw total intake for the week, but not which area would send what type of waste tomorrow — so staffing and planning stayed guesswork.

The opportunity: make EcoCycle see the waste, respect the vehicle, and predict tomorrow. That is what Muse built.

MEET THE BUILDER

What Meta Muse did on this project

Muse was not used to write a few lines of code. It worked like a product engineer on the team, in four clear steps:

STEP 1

Studied the whole app

Read the entire existing platform — screens, database, money rules and AI code — and made a full architecture map before changing anything.

STEP 2

Designed the improvements

Turned three problems into three buildable features, with clear rules for what AI may do and what must stay exact and safe.

STEP 3

Built them end-to-end

Wrote the new backend services, enforced the rules in code, and redesigned the household, agent and admin screens in the app's own design style.

STEP 4

Tested everything

Ran the full test suite plus new tests for every new feature, and fixed issues until all checks passed.

Why this matters for Meta

This is the capability on show: Muse can enter an existing, real codebase — not a blank page — understand it, improve it safely, and prove the result.

THE CHANGE AT A GLANCE

Before Muse → After Muse

Every row below is a feature Muse implemented in the working app. The next three slides show each one.

FEATURE	BEFORE	AFTER MUSE
Household request	Typed a guessed weight	Snaps a photo → gets an estimated weight range
Agent trip decision	Travelled on a guess	Sees the household's photo before travelling
Vehicle capacity	Shown, but not enforced	Live meter that locks new jobs when full
Facility planning	One weekly total	Tomorrow by area + waste type, with a briefing
Trust in the numbers	Hard to explain	Every estimate labelled with confidence

FEATURE 1 · HOUSEHOLD

Snap the waste, skip the guessing

The request screen is now photo-first. The household shows the waste instead of guessing its weight.

BEFORE

- **Guessed weight**
  A slider and a typed number, often far from reality.
- **No proof of waste**
  Agents saw only text: category and a guessed kg.
- **Wrong expectations**
  Pay and trip value were based on that guess.

AFTER — BUILT BY MUSE

- **Photo + bag count**
  Snap the waste, count the bags. That is the whole input.
- **Smart estimated range**
  Muse built an estimator: bags x waste density + photo reading + household history.
- **Confidence shown**
  Example: '8.4 kg (6.5-10.3 kg) · Medium confidence'. Honest, not fake-exact.

Result: agents travel with evidence, households stop guessing, and estimates get fairer with every verified pickup.

FEATURE 2 · COLLECTION AGENT

A vehicle that cannot overfill

Agents earn per trip. Overloading wastes fuel, time and trust. Muse made capacity a rule, not a display.

BEFORE

- **Capacity was decoration**
  A percentage on a card. Nothing stopped over-accepting.
- **Jobs won blind**
  An agent could promise more than the vehicle could hold.
- **Surprises at pickup**
  No photo, no confidence: every trip was a gamble.

AFTER — BUILT BY MUSE

- **Live capacity meter**
  Green to amber to red, with kg collected in the last hour.
- **Hard lock at full**
  Muse enforced the rule in the claim code itself: a full vehicle cannot accept new jobs.
- **Evidence on every job**
  Household photo, confidence, and 'fits your vehicle — 14 kg to spare'

Result: fewer wasted trips, no overloaded vehicles, and agents who can plan a profitable round at a glance.

FEATURE 3 · FACILITY MANAGERS

Know tomorrow, area by area

The facility decides staffing and line time every morning. Muse turned raw history into a forecast managers can act on.

BEFORE

- **One weekly total**
  'About 40 tonnes this week' — true, but not actionable.
- **No area or waste-type view**
  Which area sends plastic? Which sends organic? Unknown.
- **Planning by habit**
  Staffing followed routine, not expected arrivals.

AFTER — BUILT BY MUSE

- **Tomorrow by area**
  Expected kg, pickup count and leading waste type per area, each with confidence.
- **7-day waste mix**
  The expected stream mix for inspection and line planning.
- **Morning briefing**
  Muse built a briefing that states tomorrow's total, leading area and staffing implication.

Result: the facility staffs to expected arrivals instead of habit — the first step from reactive to predictive operations.

MUSE CAPABILITY, DEMONSTRATED

How Muse improved EcoCycle

Each capability below was demonstrated on this real project — not in a demo, but in production-style code that now runs and is tested.

✓ **Understood existing code**	Mapped 3 user surfaces, the database, the money ledger and the state machine — then built on them instead of replacing them.
✓ **Designed like a product team**	Chose ranges over fake precision, confidence labels over silent guesses, and enforcement over decoration.
✓ **Wrote full-stack code**	New Laravel services (load estimator, behaviour forecast), rule enforcement, and Livewire screens — all in the app's own patterns.
✓ **Redesigned the experience**	Photo-first request, capacity meter, evidence job cards and a forecast console — in EcoCycle's existing pine-and-clay design system.
✓ **Kept the money safe**	Estimates never pay anyone. Payment still follows the facility's verified scale weight, split 60/40 exactly as before.
✓ **Proved the work**	Ran the whole platform's tests plus new feature tests. Results are on the next slide — plain and checkable.

PROOF IT WORKS

Tested and working

After Muse's changes, the platform was run and checked. These are the measured results, in plain language:

82 / 82

Existing tests still pass

Nothing Muse changed broke the original platform.

4 / 4

New feature tests pass

Estimator, photo flow, capacity lock and forecast all tested.

0

Broken screens

Every page template compiles and renders after the redesign.

1

Unchanged money rule

Payment still follows only the verified facility scale weight.

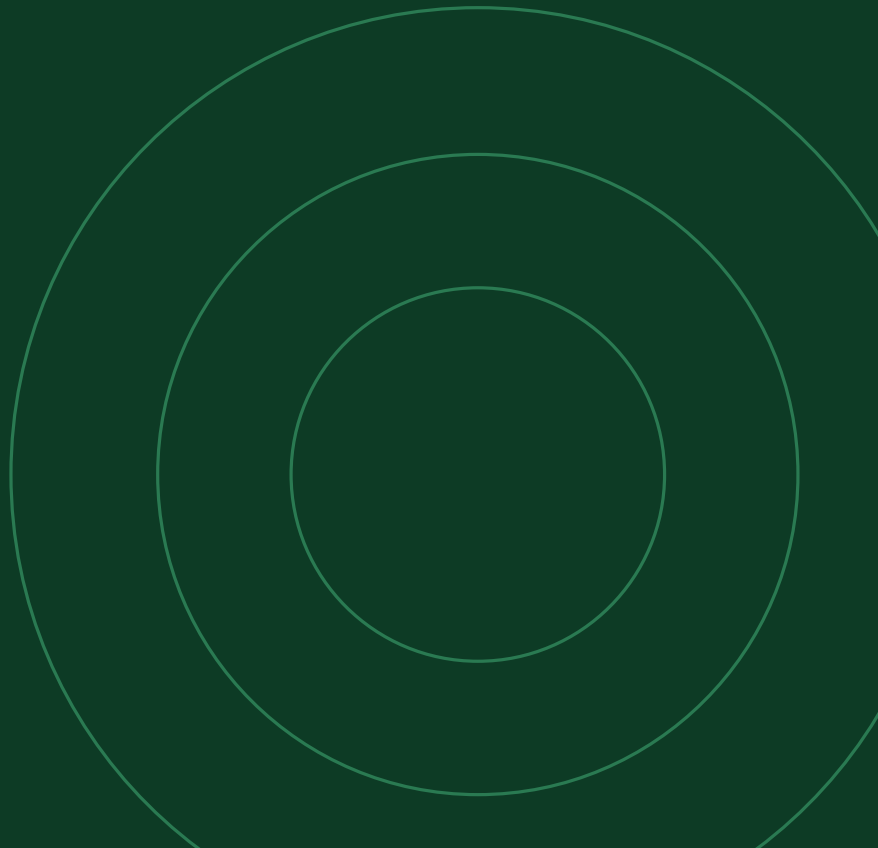
The strongest proof is the journey test: one batch still travels from a household's request, through agent collection and facility inspection, to a correct payment for both sides — now with a photo estimate at the start and a capacity check in the middle.

In short: Muse improved the app, and the app's own tests confirm nothing was broken in the process.

WHY THIS WINS

Muse did not just write code. It improved a real product.

- Households snap waste instead of guessing — faster, fairer requests.
- Agents see evidence and carry only what their vehicle holds.
- Facilities predict tomorrow by area and waste type, and staff to it.
- And the money rule never moved: the facility scale still has the last word.



EcoCycle x Meta Muse · Built, redesigned and tested by Muse in Muse Code · Thank you